

Support Insights Project Handoff

Customer Support Analytics Platform - Current Status and Step-by-Step Continuation Plan

Prepared on: 31 August 2026

1. Project Overview

Support Insights is a full-stack customer support analytics platform. The goal is to manage customer support tickets, connect them with agents and customers, track SLA rules, show dashboard analytics, and later add an intelligence layer for polarity analysis, SLA breach probability, escalation probability, anomaly detection, and recommendations.

The project is being built step by step for learning. The current focus is getting the database, backend APIs, and dashboard data flow working properly before adding authentication, frontend polish, or real AI/ML.

2. Where The Project Is Now

Area	Status
Database setup	Completed for the main foundation tables
Sample data	Added for users, customers, categories, SLA rules, tickets, messages, events, feedback, and predictions
Backend server	Created with Node.js and Express
Database connection	Created using pg and dotenv
First API	GET /api/tickets created and tested in browser
Dashboard APIs	Added to server.js in the latest step; must be restarted and tested
Frontend	Not created yet
Authentication	Not created yet
Real ML/AI service	Not created yet; prediction values are sample data for now

3. Current Folder Structure

```
Support-insights/  
  .env  
  README.md  
  backend/  
  config/  
  db.js  
  node_modules/  
  package.json  
  package-lock.json  
  server.js
```

Important note: .env contains database connection values. Do not publish passwords to GitHub.

4. Technology Stack

Layer	Technology	Purpose
Backend	Node.js + Express	Create APIs for tickets and dashboard data
Database	PostgreSQL	Store relational support data and run analytics queries

Database package	pg	Connect Node.js backend to PostgreSQL
Environment config	dotenv	Read DB credentials from .env
CORS	cors	Allow frontend to call backend later
Frontend	React	Planned next major part
ML/AI	Python service	Planned later for polarity and risk predictions

5. Database Details

Database name:

```
support_insights
```

Tables Created

Table	Purpose
users	Stores internal system users such as admin, manager, and support agents
customers	Stores customers who raise support tickets
categories	Stores ticket categories such as Payment, Login, Refund, Billing, Technical, Account
sla_rules	Stores response and resolution time limits for ticket priorities
tickets	Main table that stores customer support issues
ticket_messages	Stores customer-agent conversation messages inside a ticket
ticket_events	Stores ticket history such as created, assigned, status changed, resolved
feedback	Stores customer rating and comments after resolution
predictions	Stores polarity, SLA breach probability, escalation probability, and recommended action

Main Relationships

```
tickets.customer_id -> customers.id
tickets.agent_id -> users.id
tickets.category_id -> categories.id
ticket_messages.ticket_id -> tickets.id
ticket_events.ticket_id -> tickets.id
ticket_events.user_id -> users.id
feedback.ticket_id -> tickets.id
feedback.customer_id -> customers.id
predictions.ticket_id -> tickets.id
```

Important Term: Polarity

The project uses polarity instead of sentiment. Polarity is the technical NLP term for whether text has a positive, neutral, or negative orientation.

```
polarity = positive / neutral / negative orientation of customer text
polarity_score = confidence or strength of that polarity
```

6. Sample Data Added

- 1 internal user: Admin User
- 3 customers: Ravi Kumar, Priya Sharma, Arjun Reddy

- 6 categories: Billing, Technical, Account, Payment, Login, Refund
- 4 SLA rules: Critical, High, Medium, Low
- 3 sample tickets: Payment issue, Login issue, Refund issue
- Ticket messages for sample conversations
- Ticket events for ticket history
- Feedback for at least one resolved ticket
- Prediction rows for polarity, SLA risk, escalation risk, and recommended action

Verified Joined Ticket Output

Ticket	Customer	Category	Priority	Status	Polarity	SLA Risk	Escalation Risk
7	Ravi Kumar	Payment	High	Open	negative	0.7600	0.6900
8	Priya Sharma	Login	Medium	In Progress	neutral	0.2000	0.1500
9	Arjun Reddy	Refund	High	Open	negative	0.6400	0.7200

7. Backend Details

Current Packages

```
express
pg
dotenv
cors
```

db.js Purpose

backend/config/db.js creates a PostgreSQL connection pool. Other backend files use this pool to run SQL queries.

```
server.js -> require('./config/db') -> PostgreSQL support_insights database
```

Current APIs

API	Purpose	Status
GET /	Simple backend health message	Working
GET /api/tickets	Returns tickets joined with customer, agent, category, and prediction data	Working/tested
GET /api/dashboard/summary	Returns dashboard card metrics	Added; restart and test
GET /api/dashboard/category-stats	Returns tickets grouped by category	Added; restart and test
GET /api/dashboard/status-stats	Returns tickets grouped by status	Added; restart and test
GET /api/dashboard/priority-stats	Returns tickets grouped by priority	Added; restart and test
GET /api/dashboard/risk-tickets	Returns high-risk tickets based on SLA or escalation probability	Added; restart and test

8. Immediate Next Step

Because dashboard APIs were added in server.js, the immediate next step is to restart the backend and test every endpoint in the browser.

```
cd C:\Users\yagnesh\OneDrive\Desktop\Support-insights\backend
node server.js
```

If the server is already running, stop it first with Ctrl + C, then run node server.js again.

Test These URLs

```
http://localhost:5000/
http://localhost:5000/api/tickets
http://localhost:5000/api/dashboard/summary
http://localhost:5000/api/dashboard/category-stats
http://localhost:5000/api/dashboard/status-stats
http://localhost:5000/api/dashboard/priority-stats
http://localhost:5000/api/dashboard/risk-tickets
```

9. One-Hour Fast Plan To Complete Major Demo

Time	Task	Output
0-10 min	Restart backend and test dashboard APIs	Backend API layer verified
10-20 min	Fix any API errors if they appear	Stable backend responses
20-30 min	Create React frontend project	Frontend folder created
30-45 min	Build dashboard screen with summary cards and tickets table	Usable dashboard UI
45-55 min	Connect frontend to backend APIs	Real database data visible in UI
55-60 min	Quick polish and final test	Demo-ready major project flow

10. Frontend Plan

After backend APIs are tested, create a React frontend. The first screen should be the actual dashboard, not a landing page.

Dashboard Should Show

- Summary cards: total tickets, open tickets, in-progress tickets, resolved tickets, high-priority tickets
- Risk cards: average SLA breach probability, average escalation probability, negative polarity tickets
- Tickets table using GET /api/tickets
- Small sections for category stats, status stats, priority stats
- High-risk tickets list using GET /api/dashboard/risk-tickets

11. Later Project Roadmap

Phase	Work
Backend APIs	Customers, users, categories, create/update tickets, messages, feedback
Authentication	Register/login, password hashing, JWT, role-based access
Frontend	Dashboard, ticket list, ticket details, create ticket, update status
Analytics	Category trends, SLA compliance, agent performance, channel stats
AI/ML	Real polarity analysis, SLA breach prediction, escalation prediction

Recommendations	Rule-based actions based on risk and workload
Deployment	Frontend, backend, and database hosting
Documentation	Report, diagrams, screenshots, interview explanation

12. Notes For Another AI

- Do not restart the project from scratch. Continue from the existing backend and database.
- The database already contains working sample data and relationships.
- The current backend file is backend/server.js.
- The database connection file is backend/config/db.js.
- The .env file is in the project root, not inside backend.
- The first working API is GET /api/tickets.
- Next practical step is to test dashboard APIs, then create React frontend.
- Use polarity, not sentiment, in project explanations and API fields.
- Avoid adding advanced ML before the basic dashboard works.

13. Suggested Completion Order

1. Verify backend APIs
2. Create frontend
3. Connect dashboard summary cards
4. Connect tickets table
5. Add category/status/priority stats sections
6. Add high-risk tickets section
7. Add create-ticket API and form
8. Add update ticket status API
9. Add authentication
10. Add real ML/AI later

14. Current Completion Estimate

The project is approximately 25-30% complete. The database and first backend API are complete. The major missing visible part is the frontend dashboard. Once the React dashboard consumes the APIs, the project will feel much more complete and demo-ready.

15. Resume Summary

Support Insights has a PostgreSQL database with core tables and sample data. Node.js/Express backend is connected to PostgreSQL. GET /api/tickets works. Dashboard API routes have been added and should be restarted/tested next. After that, build a React dashboard that displays API data from the backend.